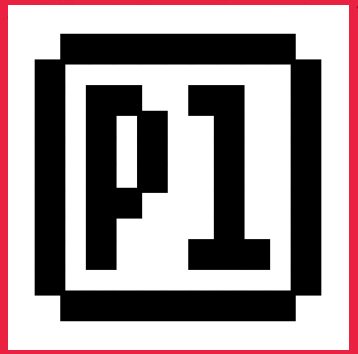


Git + Ejercicios

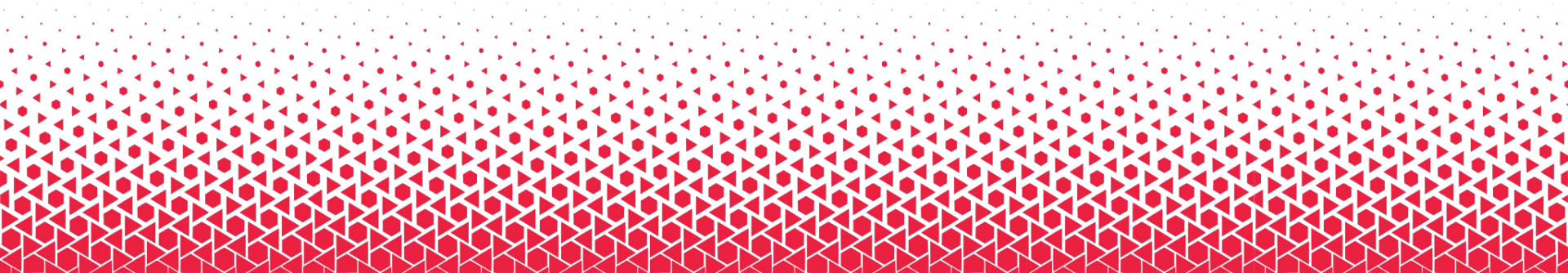
Mauro Fermín
mafermin@unrn.edu.ar

UNRN
Universidad Nacional
de **Río Negro**

r33

04 / 2026

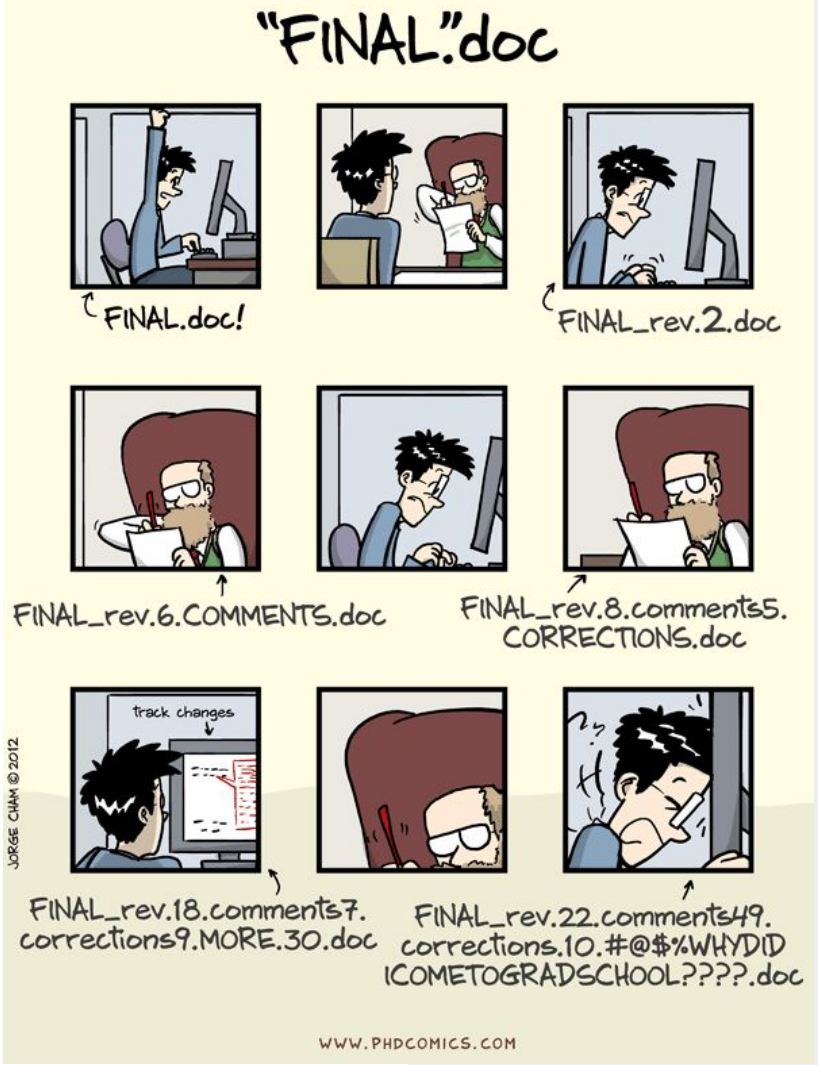
Introducción al Control de versiones



¿Qué es?

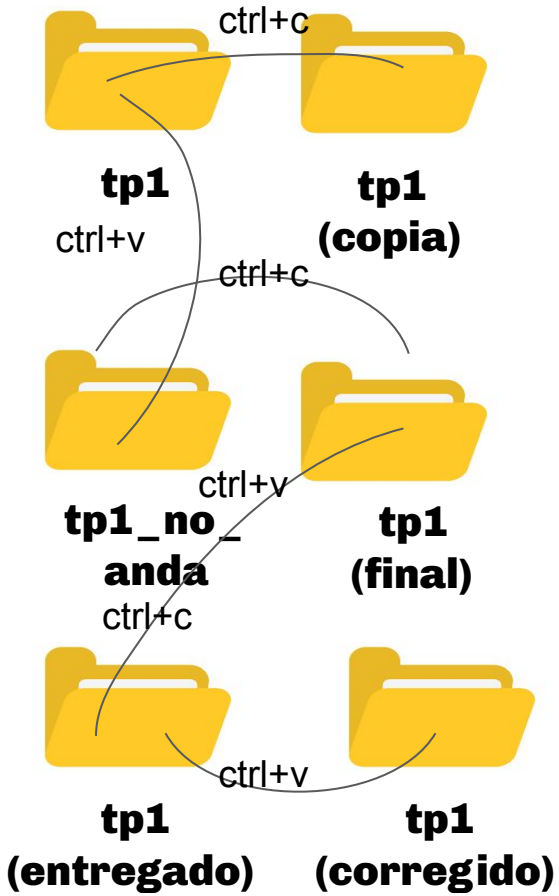
Los niveles del control de versiones

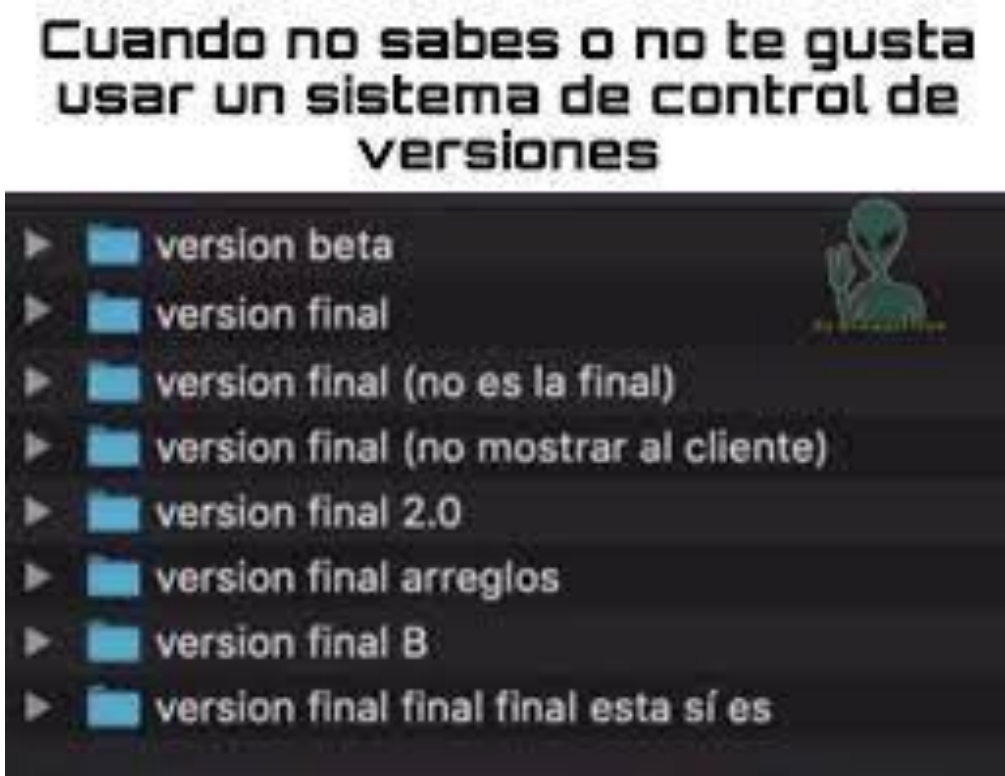
Nivel 0


"FINAL".doc
FINAL.doc!
FINAL_rev.2.doc
FINAL_rev.6.COMMENTS.doc
FINAL_rev.8.comments5.
CORRECTIONS.doc
track changes
FINAL_rev.18.comments7.
corrections9.MORE.30.doc
FINAL_rev.22.comments49.
corrections.10.#@$%WHYDID
ICOMETOGRADSCHOOL????.doc
JORGE CHAM © 2012
WWW.PHDCOMICS.COM

Nivel 0

¿Quién no ha hecho esto?

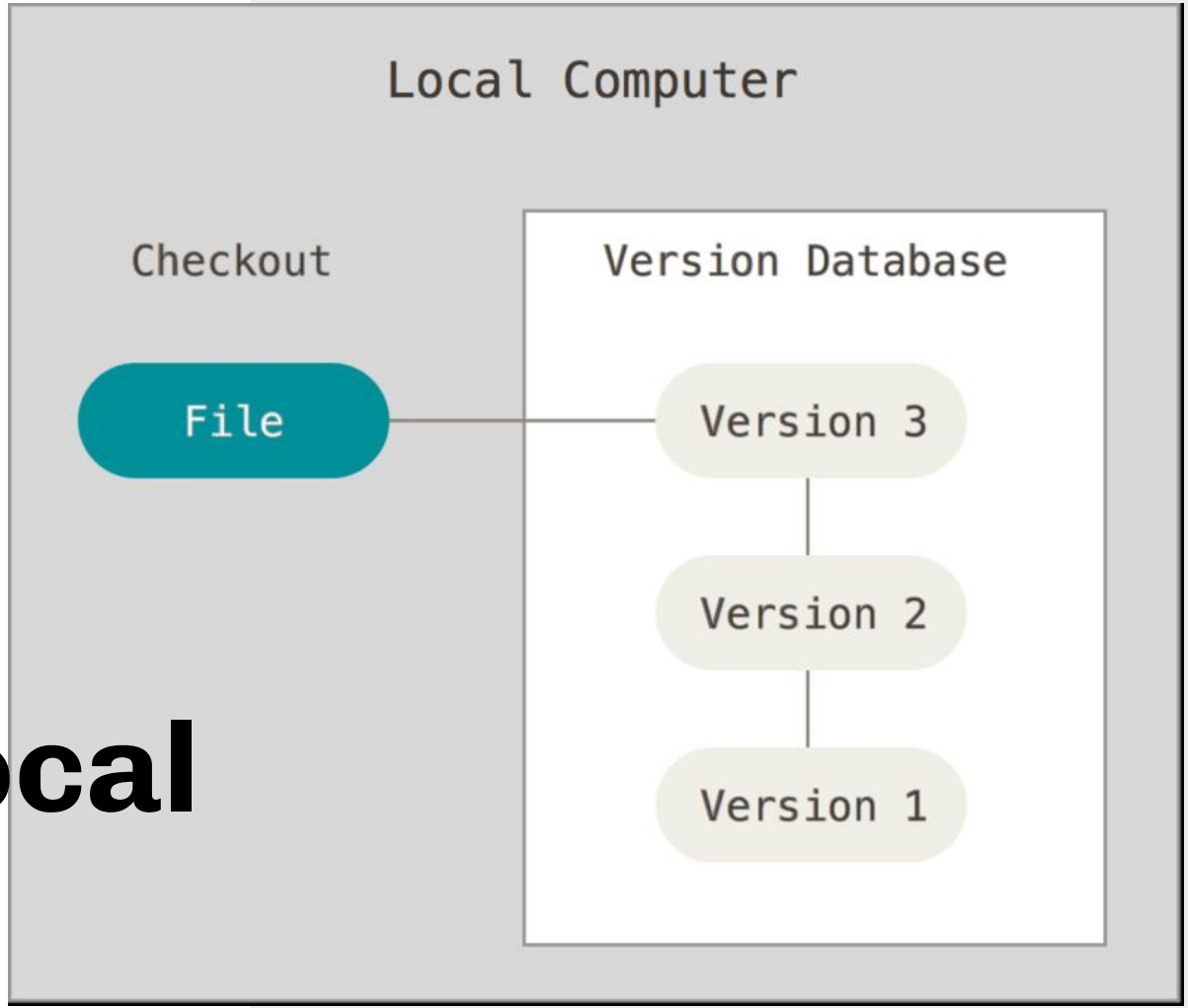


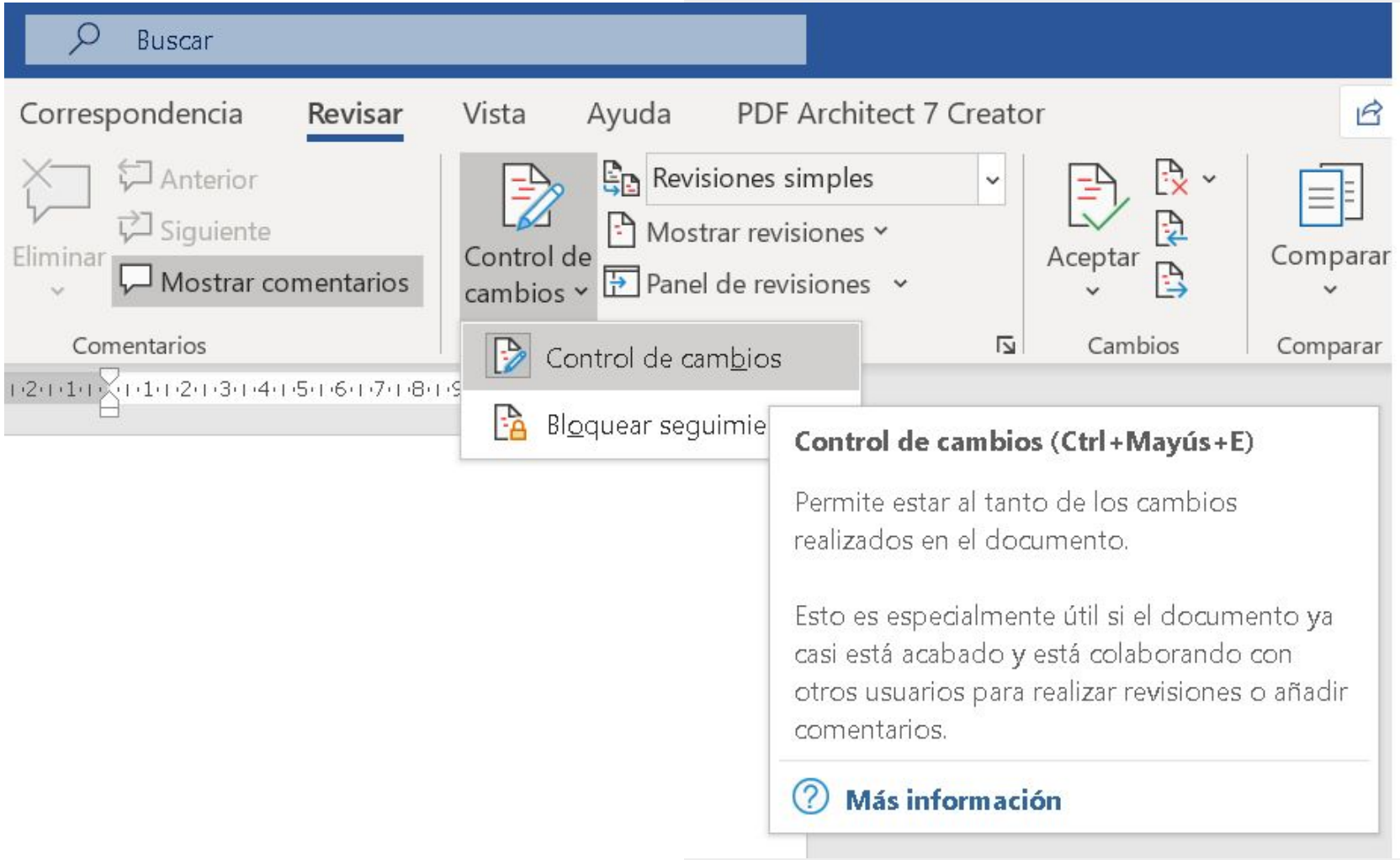


Nivel 0

Duplicar manualmente archivos

Nivel 1
Sistema local

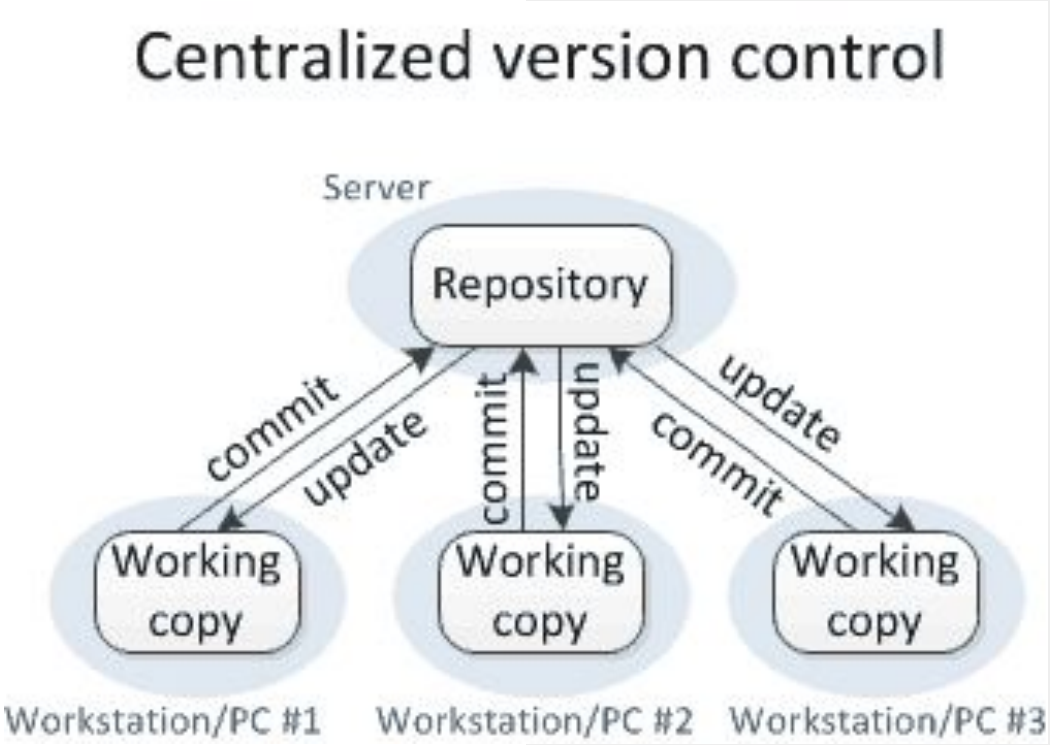



Buscar
Correspondencia
Revisar
Vista
Ayuda
PDF Architect 7 Creator
Anterior
Siguiente
Eliminar
Mostrar comentarios
Comentarios
Control de cambios
Revisiones simples
Mostrar revisiones
Panel de revisiones
Aceptar
Cambios
Comparar
Comparar
Control de cambios
Bloquear seguimie
Control de cambios (Ctrl+Mayús+E)
Permite estar al tanto de los cambios realizados en el documento.
Esto es especialmente útil si el documento ya casi está acabado y está colaborando con otros usuarios para realizar revisiones o añadir comentarios.
Más información

¿Y el trabajo en equipo?

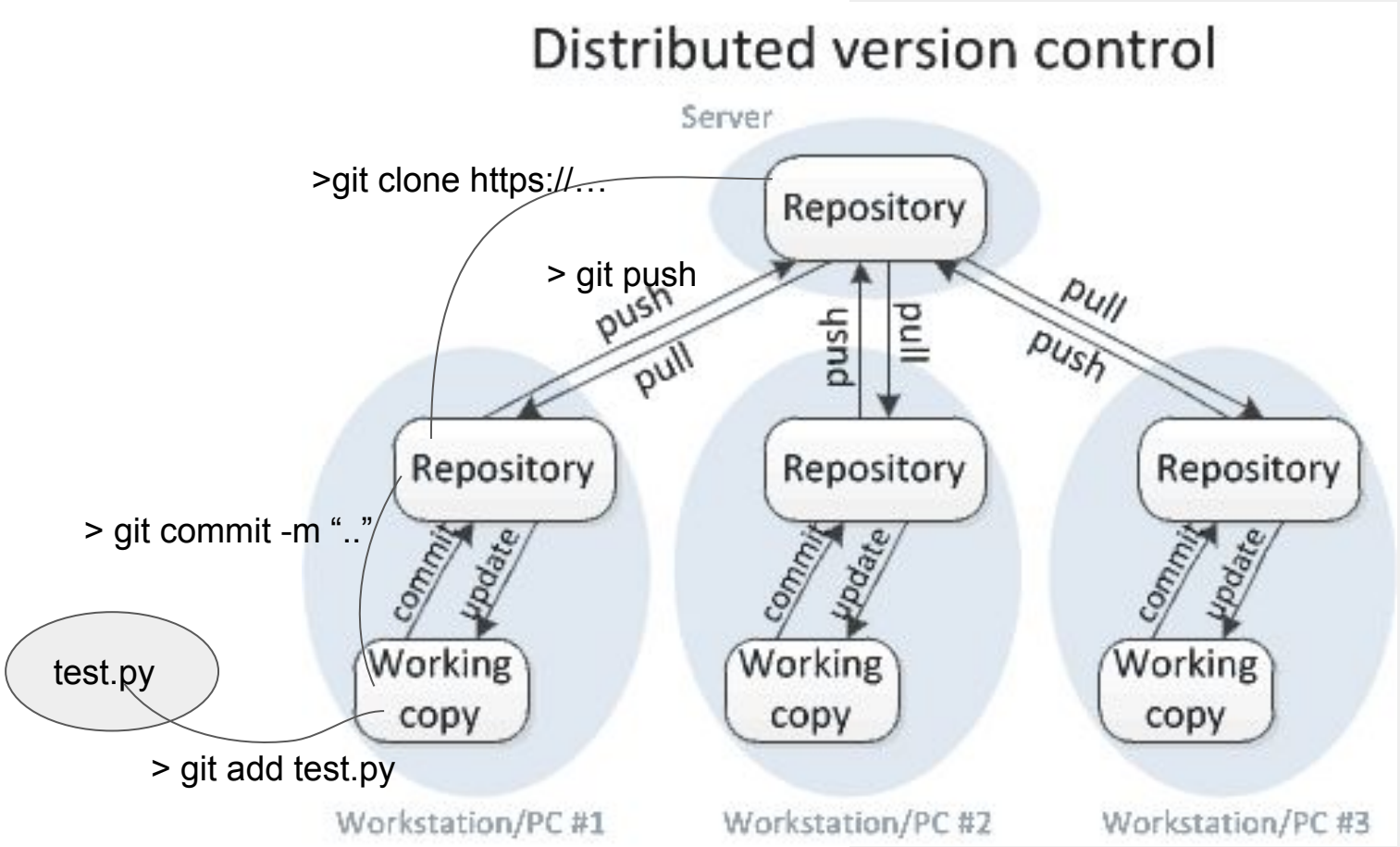
Nivel 2

archivos-historial-permisos



¿Qué pasa si se rompe el servidor?

Nivel 3


Distributed version control
Server
>git clone https://…
Repository
> git push
push
pull
push
pull
pull
push
Repository
Repository
Repository
> git commit -m “..”
commit
update
commit
update
commit
update
test.py
Working copy
Working copy
Working copy
> git add test.py
Workstation/PC #1
Workstation/PC #2
Workstation/PC #3

La historia completa está en cada repositorio

La comunicación es libre entre repositorios

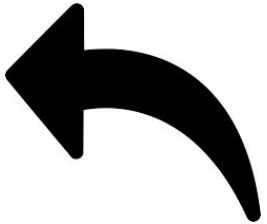
en particular vamos a ver: GIT

Eso nos lleva a qué es el control de versiones

Y lo podemos resumir en dos aspectos principales

1

Guarda todos los cambios



En términos simples;

Un deshacer infinito

2

Combina cambios con los de otros*



No es substituto a la comunicación y organización de equipo

Hay otras herramientas que se encargan de eso

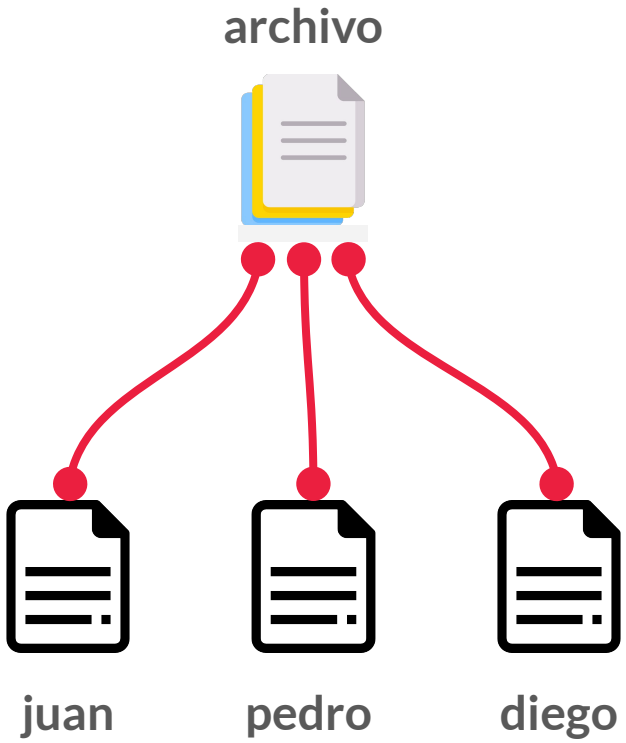

¿Preguntas?

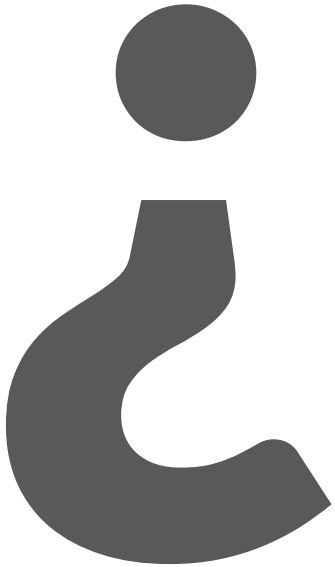
Ya para trabajar solos nos da una mano

Pero *realmente*
ayudan en equipo

Supongamos que tenemos un equipo de tres

Y tenemos un trabajo que terminar

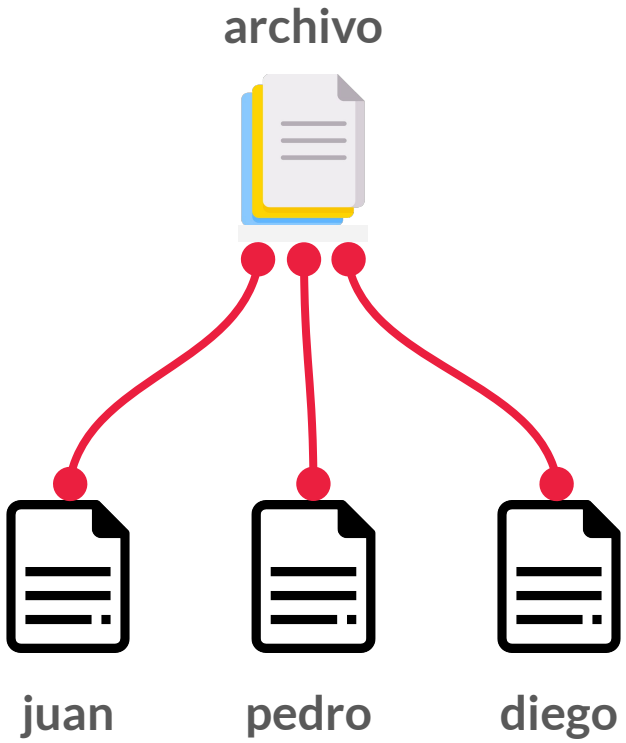




Quien tiene la última versión

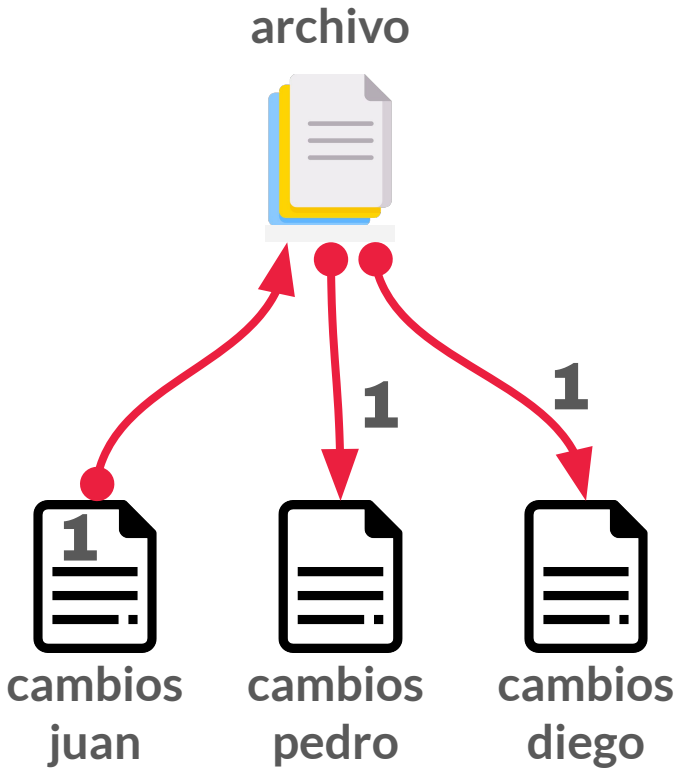


Y tenemos un trabajo que terminar

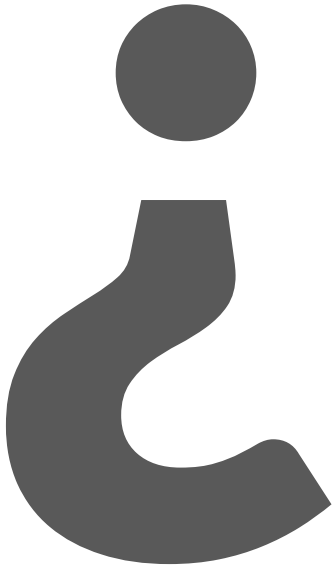


Pero...

Juan hace cambios en el archivo...



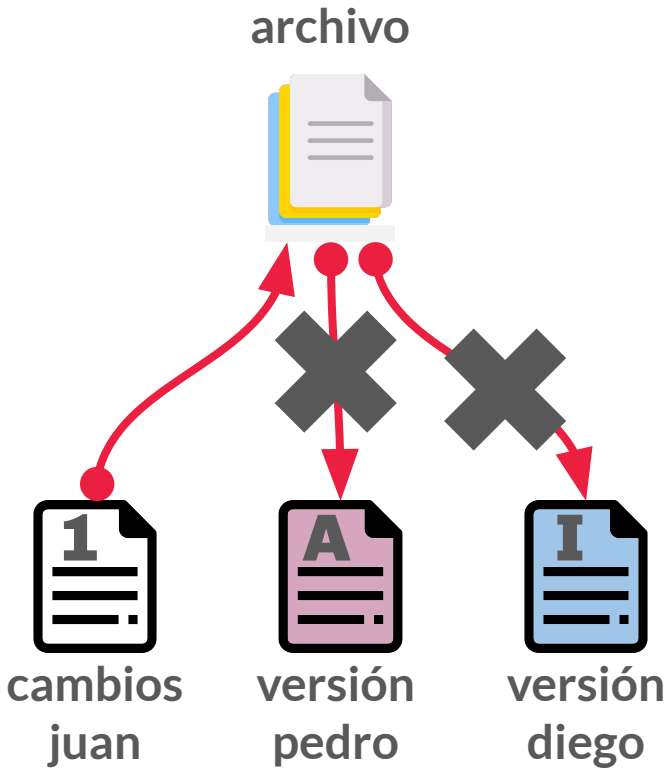
Y este se los hace llegar al resto

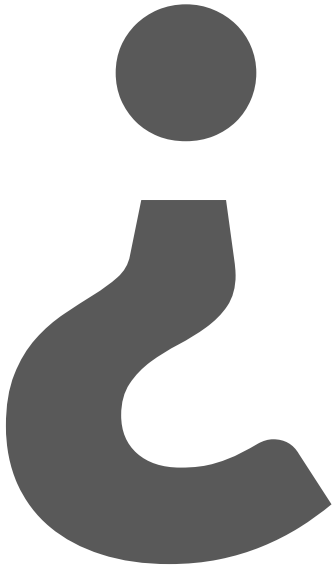


Qué pasa si el archivo estaba modificado



¿Qué pasa si Pedro y Diego ya tenían modificaciones?

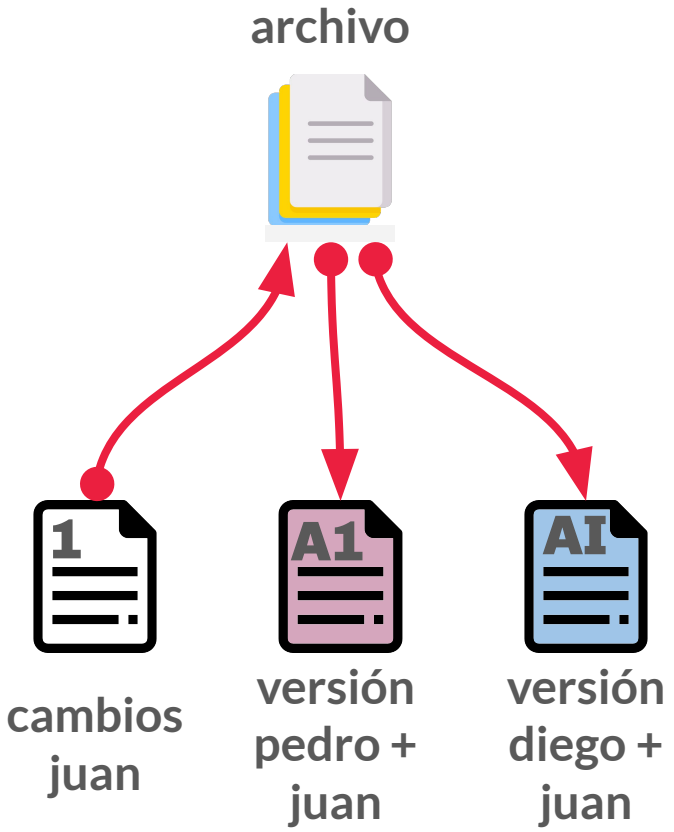




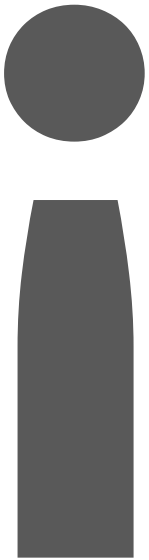
Como obtenemos la *'última'* versión



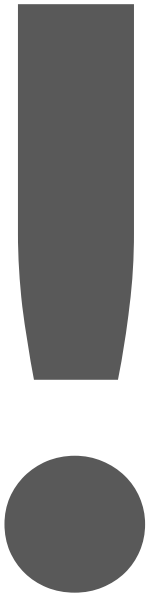
¡Cada uno va a tener una versión diferente!



¿y ahora?



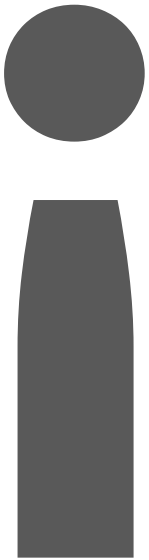
Y esto son solo *tres*



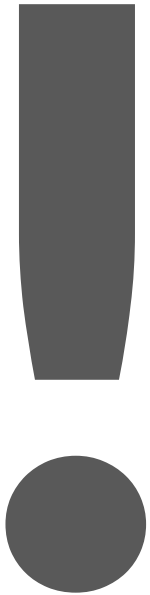
Este es el gran problema que facilita* resolver el control de versiones



No lo resuelve, pero guardar todo, ¡ayuda mucho!



Estas herramientas aportan mucho más

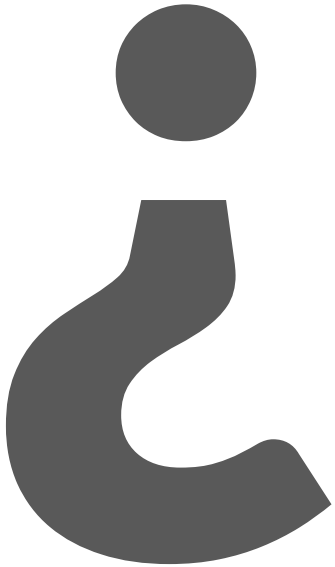


- registra cambios
- comparte trabajo
- *une* trabajos
- guarda la historia
- ayuda a hacer backups

y
en particular

Control de Versiones

con git



Qué es git



git

Git es un software de control de versiones distribuido, orientado al desarrollo de software

Git es un *software* de **control de versiones** diseñado por *Linus Torvalds*, pensando en la eficiencia, la confiabilidad y compatibilidad del mantenimiento de versiones de aplicaciones cuando estas tienen un gran número de archivos de código fuente. Su propósito es llevar **registro de los cambios** en archivos de computadora incluyendo **coordinar el trabajo que varias personas** realizan sobre archivos compartidos en un **repositorio de código**.

fuente: https://es.wikipedia.org/wiki/Git



A

Estar orientado al desarrollo implica que los archivos debieran* ser de texto

B

Cada espacio de trabajo es una copia completa del repositorio



Es una herramienta compleja, por lo que vamos a ver una pequeña parte de lo que es posible y permite.


¿Preguntas?





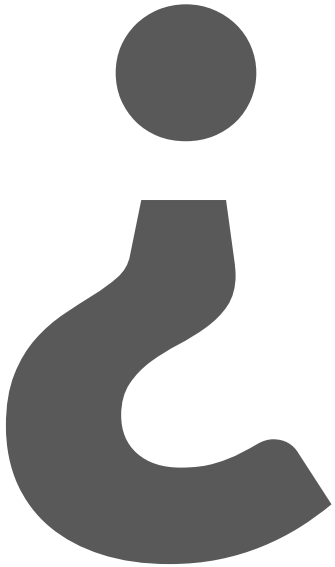
¿Qué es Git?





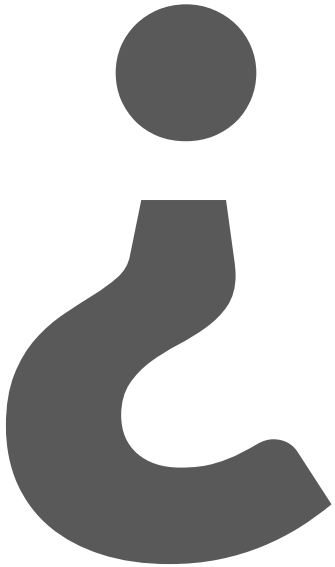
GitHub

UNRN Universidad Nacional de Río Negro



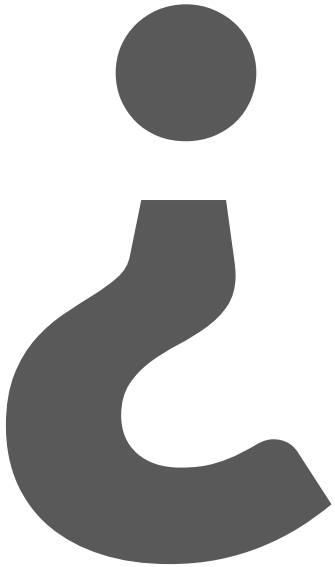
Qué es





Que nos ofrece

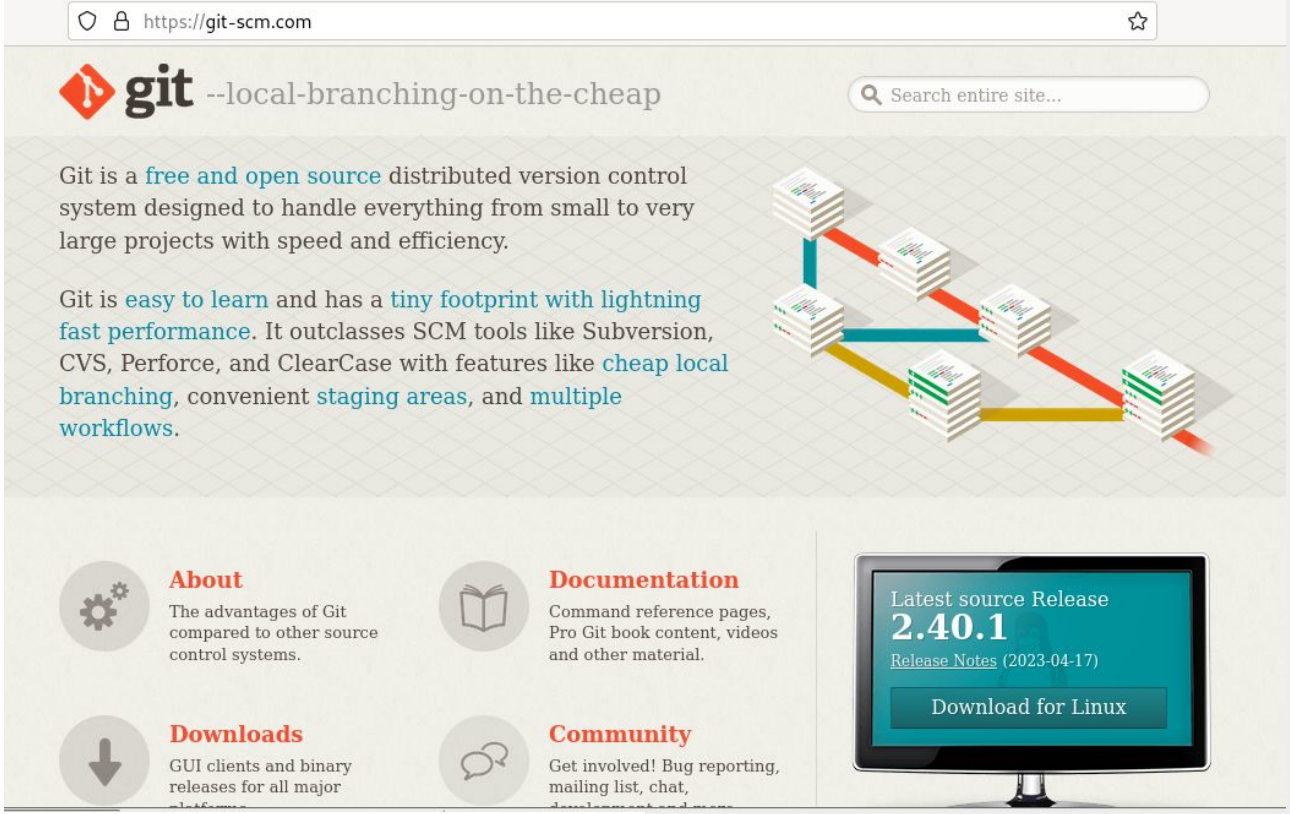




Como lo utilizaremos



Instalación



Instalación en Linux

Si quieres instalar Git en Linux a través de un instalador binario, en general puedes hacerlo mediante la herramienta básica de administración de paquetes que trae tu distribución. Si estás en Fedora por ejemplo, puedes usar yum:

```
$ yum install git
```

Si estás en una distribución basada en Debian como Ubuntu, puedes usar apt-get:

```
$ apt-get install git
```

Para opciones adicionales, la página web de Git tiene instrucciones de instalación en diferentes tipos de Unix. Puedes encontrar esta información en http://git-scm.com/download/linux.

Pero primero

Necesitamos crear una cuenta

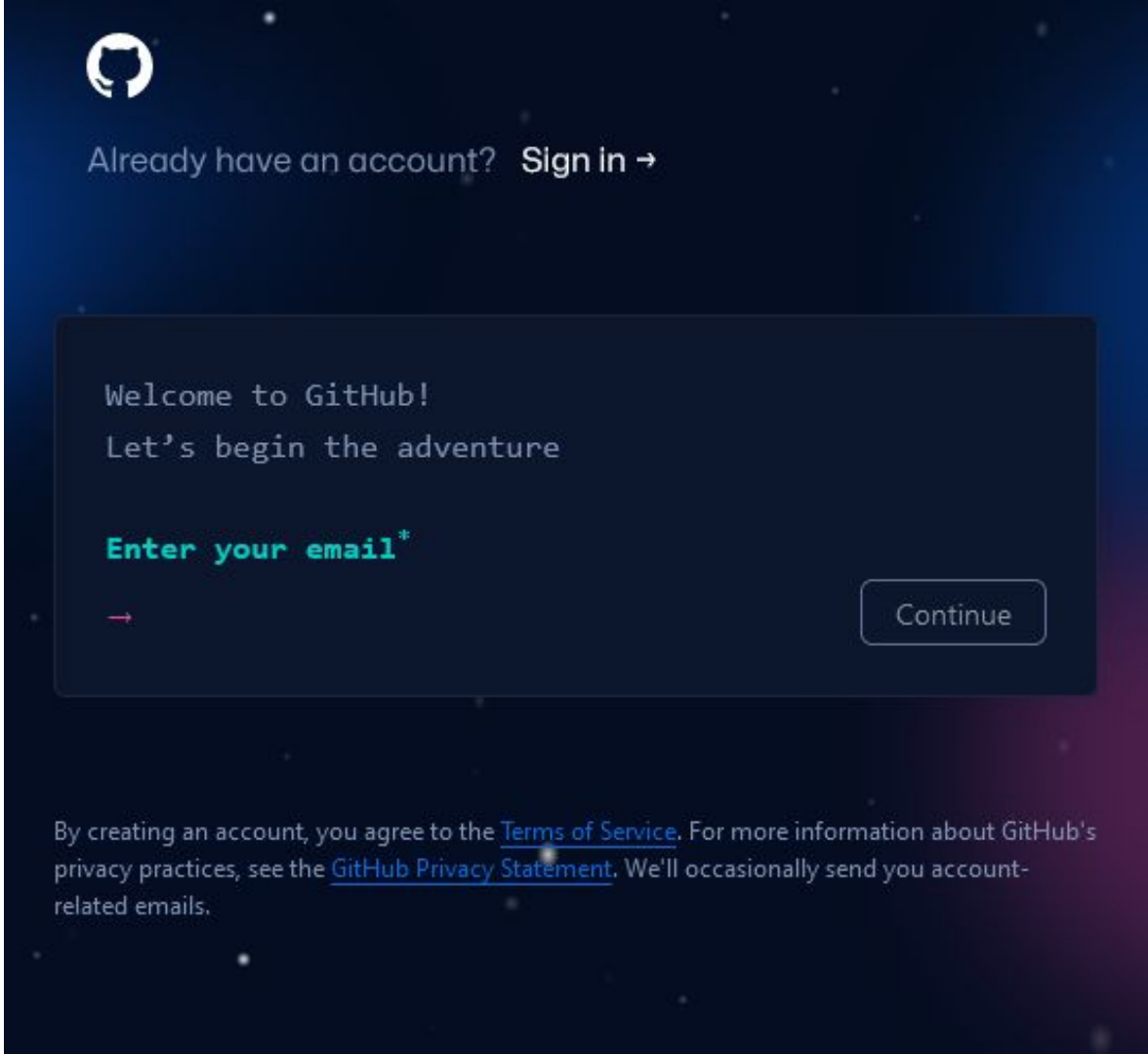
github.com/signup

Indicando:

- email
- contraseña
- nombre de usuario

y cuando terminen

Registrensé acá



El formulario es necesario para conectar todo



¡Complétenlo lo antes posible!

En un rato vemos en detalle las herramientas de GitHub

Instalación

Instalación en GNU/Linux (Derivados de Debian)

```
$> sudo apt install git
```

Descarga desde

git-scm.com/download/win

Y ejecuten el instalador

Revisen el apunte para detalles en este tema

Instalen el cliente de GitHub también

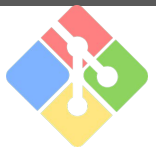
cli.github.com

Para simplificar la configuración inicial

Configuración base

Configurando nuestra identidad

En una terminal GitBash



```
git config --global user.name “Completar con mi nombre”
git config --global user.email mi.correo@electronico.com
```

El correo *tiene* que coincidir con el que usen en GitHub

Ya que vamos a usar GitHub, configuración interactiva

gh auth login

```
🚀 ~ gh auth login
? What account do you want to log into?  [Use arrows to move, type to filter]
> GitHub.com
  GitHub Enterprise Server
```

1

```
- Logging into github.com
? How would you like to authenticate?  [Use arrows to move, type to filter]
> Login with a web browser
  Paste an authentication token
```

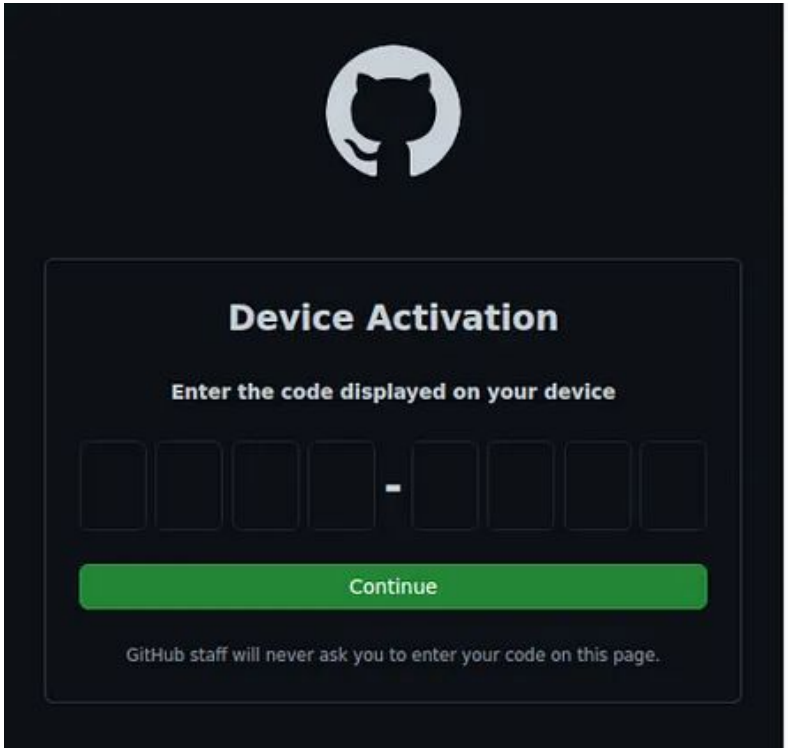
2

```
! First copy your one-time code: ████-████
- Press Enter to open github.com in your browser...
```

3

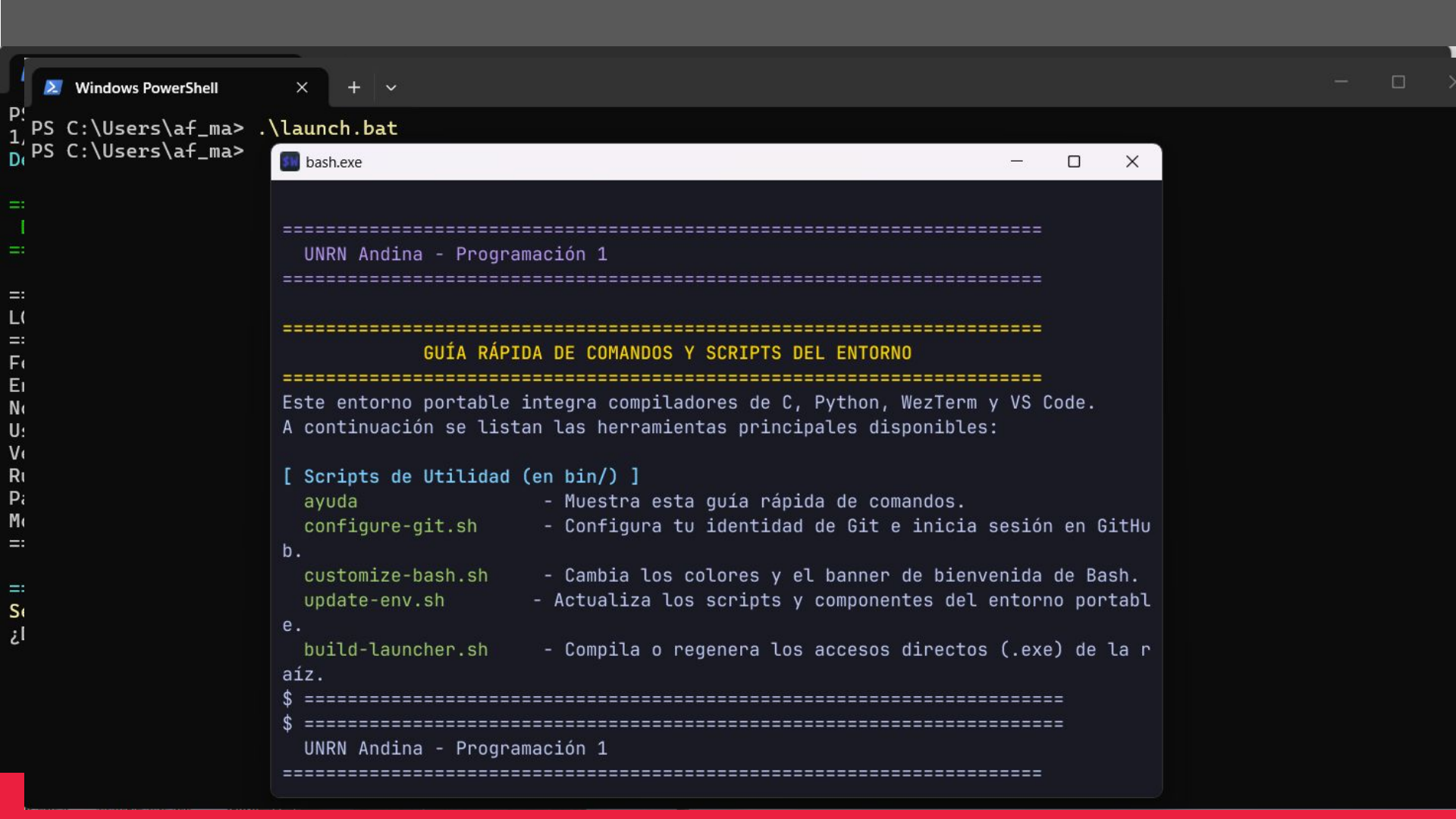
Esto simplifica la parte de seguridad

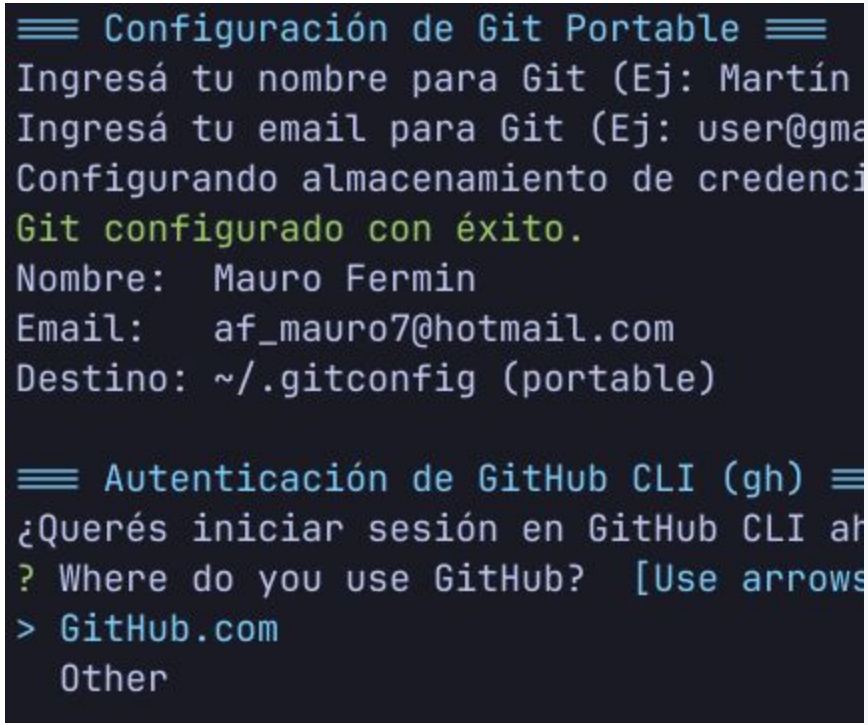
Inicio de sesión con el navegador

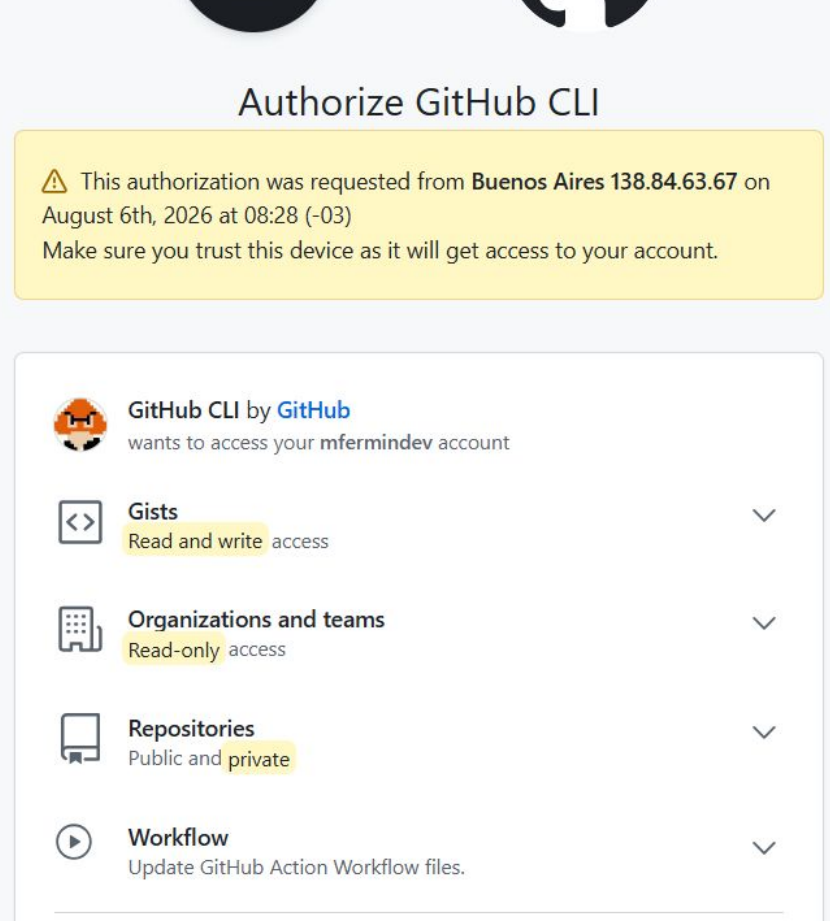




Peguen el código que aparece en la consola


Windows PowerShell
PS C:\Users\af_ma> .\launch.bat
PS C:\Users\af_ma>
bash.exe
UNRN Andina - Programación 1
GUÍA RÁPIDA DE COMANDOS Y SCRIPTS DEL ENTORNO
Este entorno portable integra compiladores de C, Python, WezTerm y VS Code.
A continuación se listan las herramientas principales disponibles:
[Scripts de Utilidad (en bin/)]
ayuda - Muestra esta guía rápida de comandos.
configure-git.sh - Configura tu identidad de Git e inicia sesión en GitHub.
customize-bash.sh - Cambia los colores y el banner de bienvenida de Bash.
update-env.sh - Actualiza los scripts y componentes del entorno portable.
build-launcher.sh - Compila o regenera los accesos directos (.exe) de la raíz.
$ ==========
$ ==========
UNRN Andina - Programación 1


Configuración de Git Portable
Ingresá tu nombre para Git (Ej: Martín
Ingresá tu email para Git (Ej: user@gma
Configurando almacenamiento de credenci
Git configurado con éxito.
Nombre: Mauro Fermin
Email: af_mauro7@hotmail.com
Destino: ~/.gitconfig (portable)
Autenticación de GitHub CLI (gh)
¿Querés iniciar sesión en GitHub CLI ah
? Where do you use GitHub? [Use arrows
> GitHub.com
Other


Authorize GitHub CLI
This authorization was requested from Buenos Aires 138.84.63.67 on August 6th, 2026 at 08:28 (-03)
Make sure you trust this device as it will get access to your account.
GitHub CLI by GitHub
wants to access your mfermindev account
Gists
Read and write access
Organizations and teams
Read-only access
Repositories
Public and private
Workflow
Update GitHub Action Workflow files.

Y para terminar

```
✓ Authentication complete. Press Enter to continue...
```

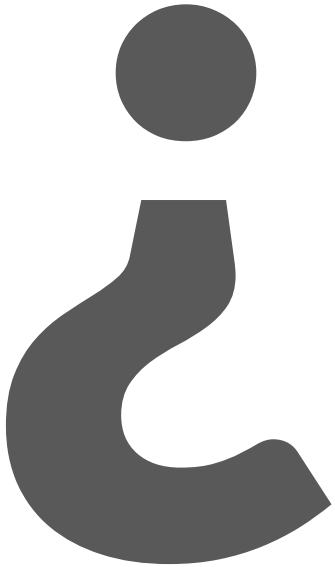
```
gh auth setup-git
```

```
gh auth status
```

```
github.com
  ✓ Logged in to github.com account martinvilu (keyring)
  - Active account: true
  - Git operations protocol: https
  - Token: gho_************************************
  - Token scopes: 'gist', 'read:org', 'repo', 'workflow'
```

Y status para confirmar que esté todo bien


¿Preguntas?



Como funciona esto

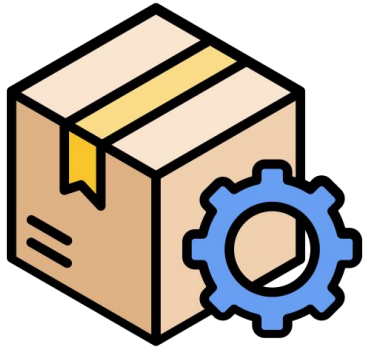


agregar

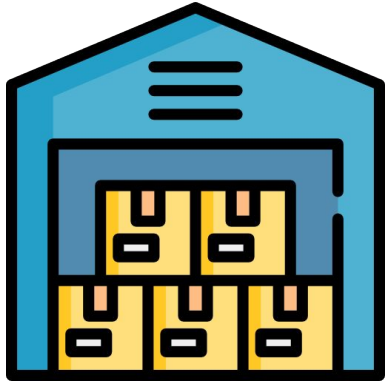
guardar*



espacio de trabajo



área de empaque

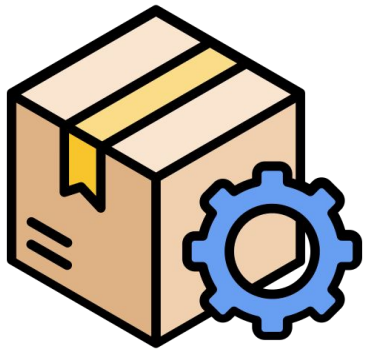


repositorio



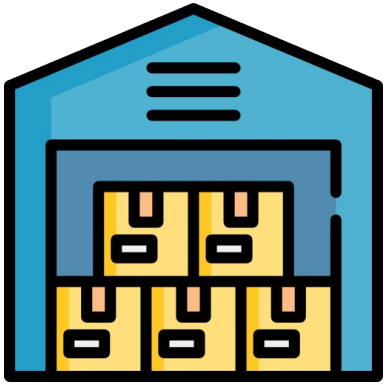
espacio de trabajo





área de empaque





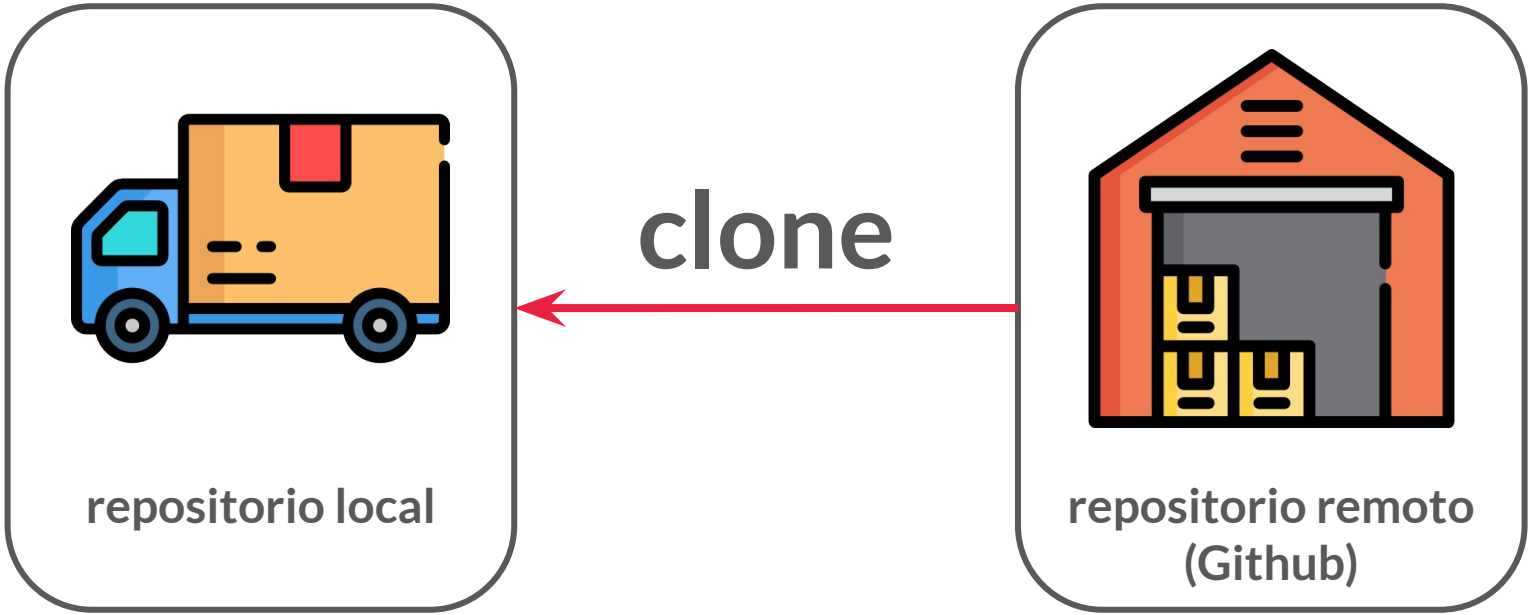
repositorio



Pasando a la terminología *apropiada*...

Paso a paso

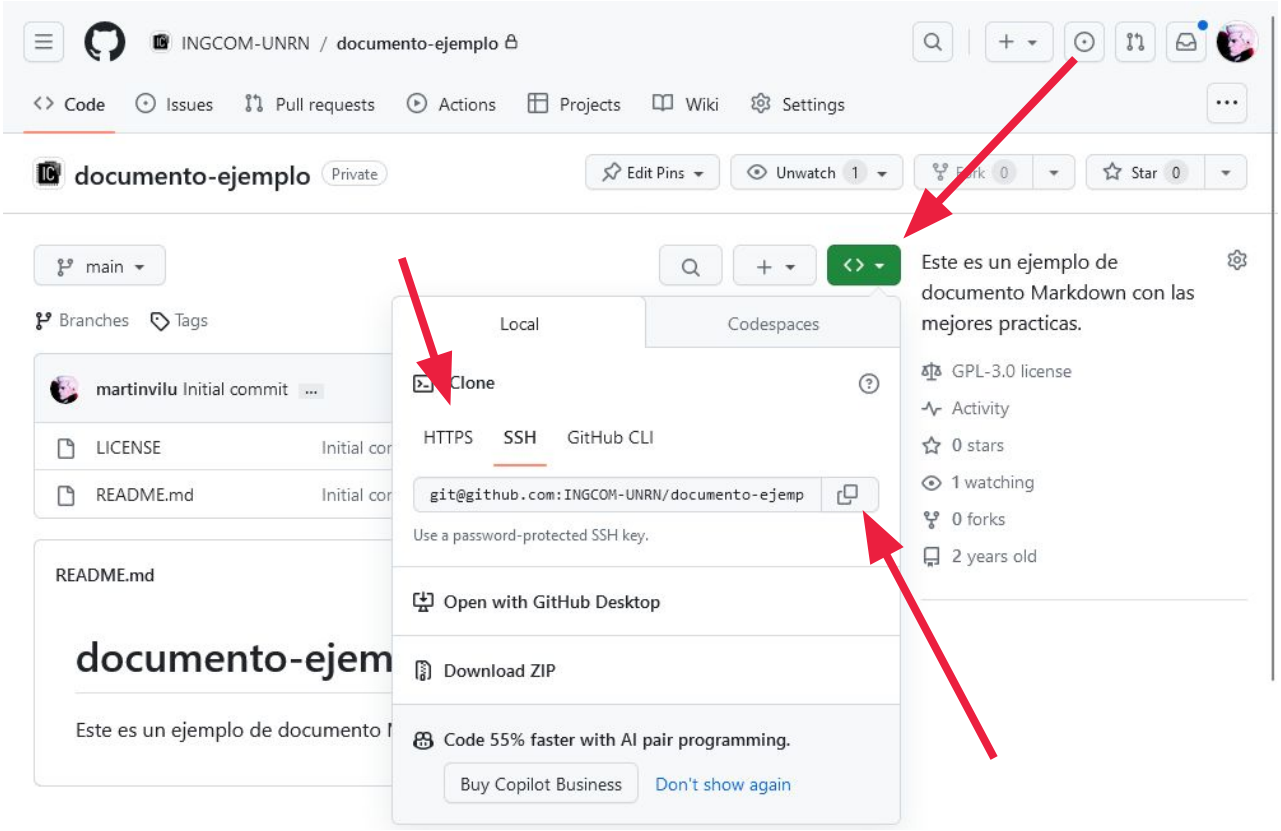
y conceptos


clone
repositorio local
repositorio remoto
(Github)

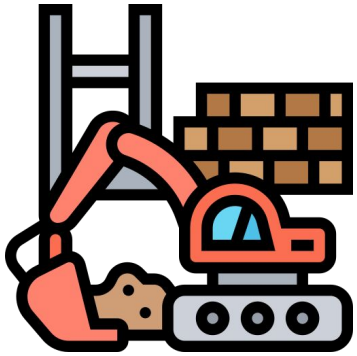
`git clone <url>`

Descarga un repositorio en nuestra computadora

```
git clone
https://github.com/INGCOM-UNRN/documento-ejemplo
```



Como obtener la dirección para usar en `git clone`



construcción del repositorio local

no hace falta para las prácticas

`git init`

Es más cómodo crearlo en Github y clonarlo

`git init <nombre>`

Crea un repositorio vacío y sin conexiones con el nombre indicado.

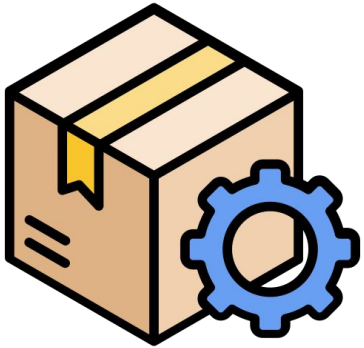


add

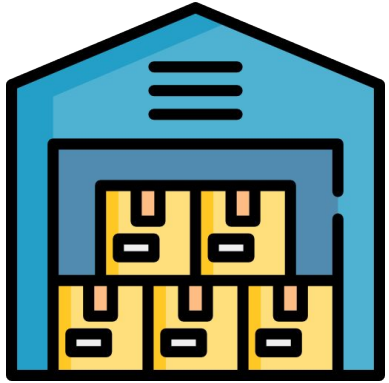
commit



working copy



staging area



local repository

workflow básico

Ciclo de trabajo normal add/commit

`git add <archivo>`

Envía una copia del archivo al área de empaque.

`git commit -m "mensaje"`

Guarda los archivos del área de empaque agregando el mensaje indicado

{

Tanto el working copy como el staging area forman parte del repositorio

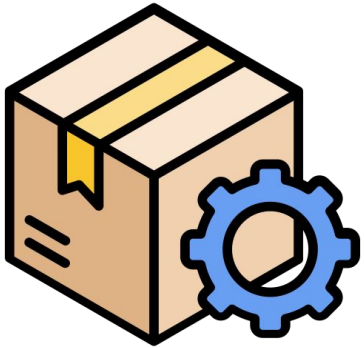
}

add

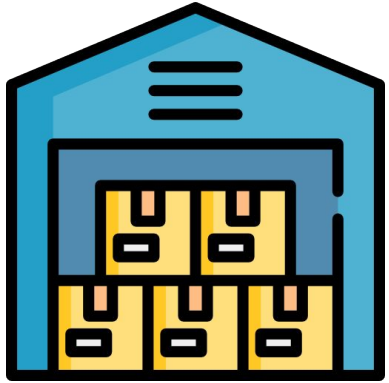
commit



working copy



staging area



local repository

reset

workflow básico

Ciclo de trabajo normal add/commit/reset

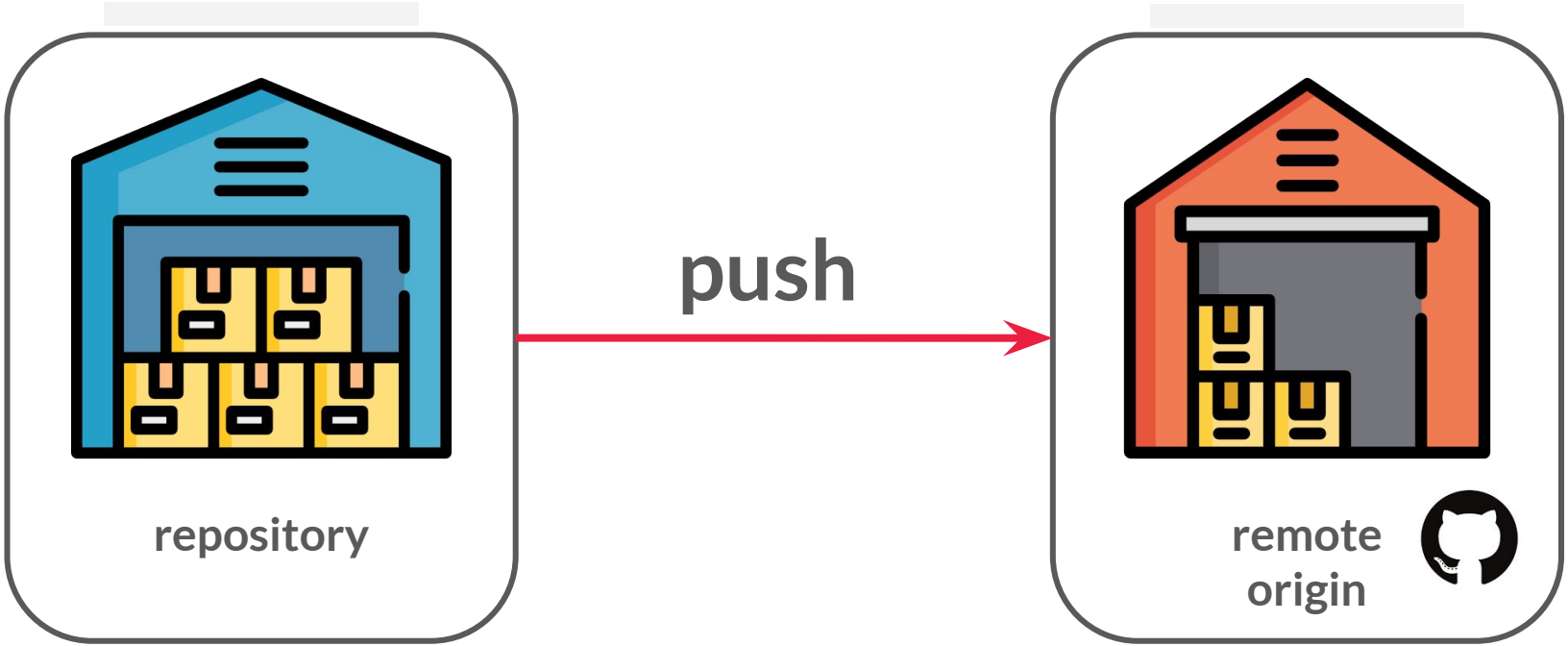
`git reset <archivo>`

Saca el archivo del staging área

`git status`

Nos indica el estado de los archivos en el repositorio, indicando que cambió relativo al último commit.

Solo falta compartir los cambios


push
repository
remote
origin

`git push`

Comparte los cambios con el repositorio en la dirección de origen*.

*`origin`, en la terminología oficial

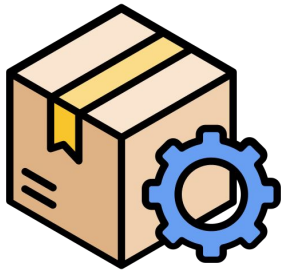
add

commit

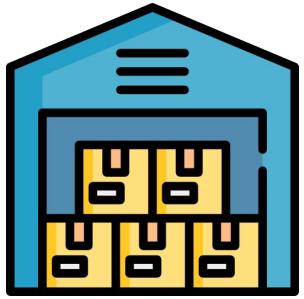
push



working copy



staging area



repository



remote

workflow básico

Ciclo de trabajo completo

El algoritmo

Mientras queden ejercicios por resolver:

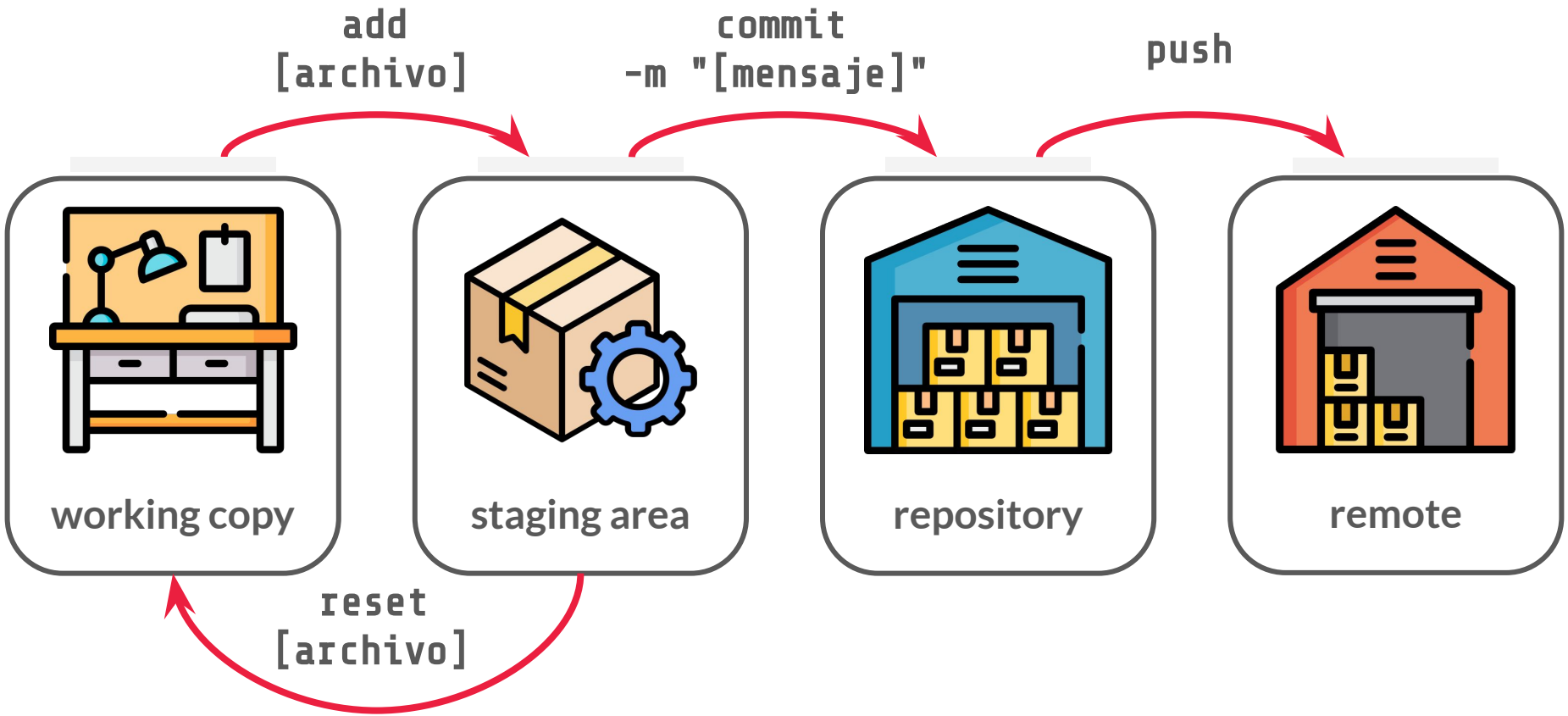
1. Se modifican los archivos
2. Se agregan los archivos al área de empaque

   ```
   git add [archivos]
   ```
3. Se cierra el paquete

   ```
   git commit -m “Descripcion breve de los cambios”
   ```

Y al terminar de trabajar, compartan los cambios con

```
git push
```



workflow básico

Ciclo de trabajo normal add/commit/push

El algoritmo

Mientras queden ejercicios por resolver:

1. Se modifican los archivos
2. Se agregan los archivos al área de empaque

   ```
   git add [archivos]
   ```

   a. Si nos arrepentimos de agregar ese archivo al area de empaque, lo sacamos con:

      ```
      git reset [archivos]
      ```

3. Se cierra el paquete

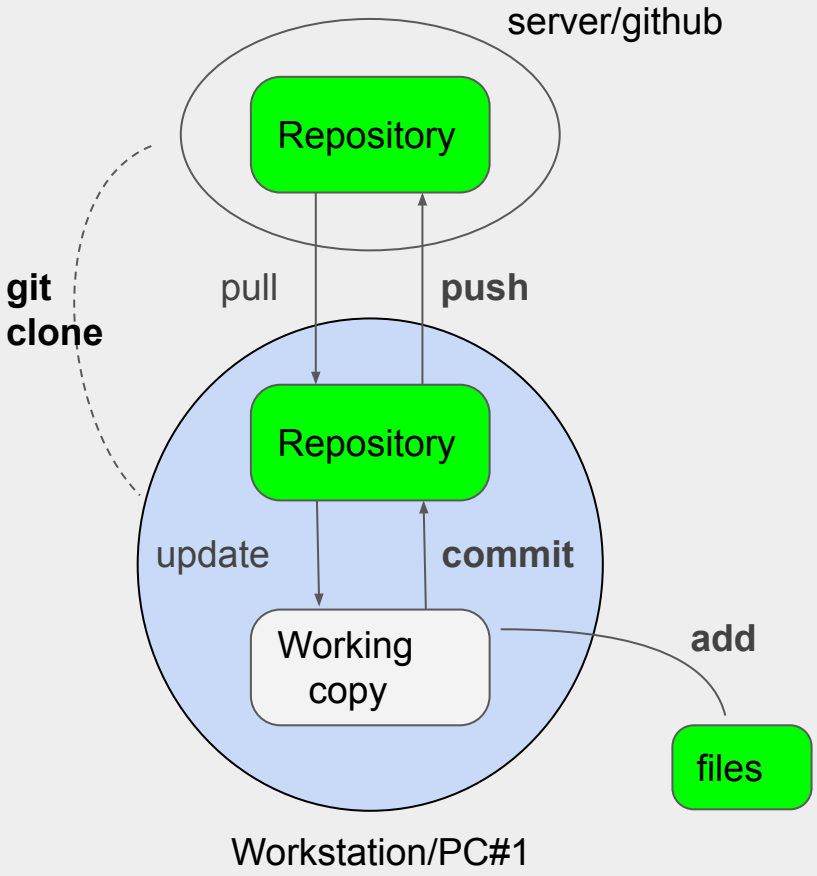
   ```
   git commit -m “Descripción breve de los cambios”
   ```

Y al terminar de trabajar, compartan los cambios con

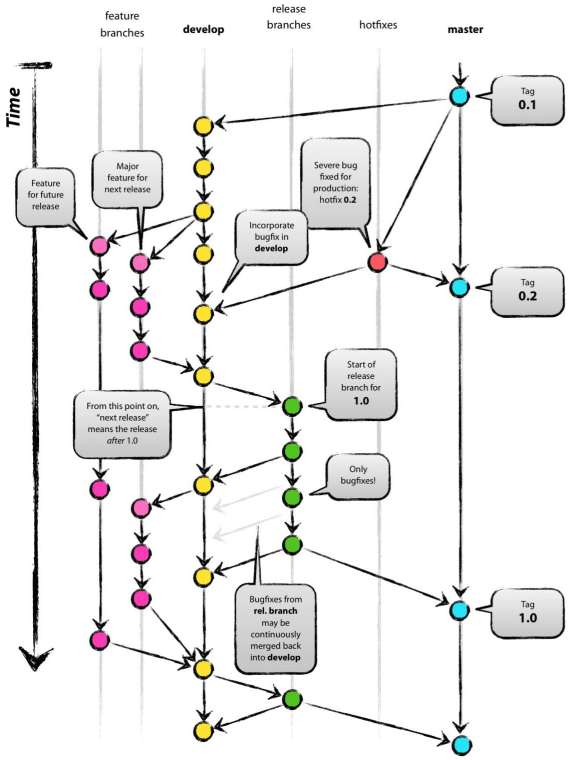
```
git push
```

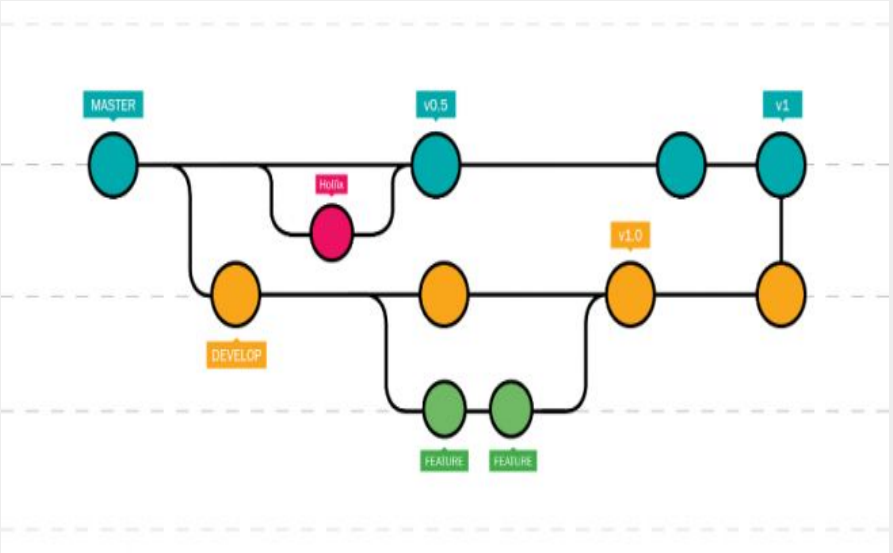
Pasos repositorio remoto -> local -> remoto

1> Crear repositorio remoto en gitlab, github
2> Clonamos el repositorio en nuestro directorio local (**git clone <url>**)
3> Trabajamos en nuestro directorio local. (creación/ modificación/ eliminamos archivos / líneas de textos).
(**git status**) ver el estado del repositorio
4> agregamos nuestras modificaciones al área de trabajo (**git add <files>**)
5> guardamos un punto de restauración en nuestro repositorio con un comentario. (**git commit -m "mensaje"**)
6> …
7> Actualizamos el repositorio remoto. (**git push** <url>)



Branches


feature branches
develop
release branches
hotfixes
master
Time
Tag 0.1
Feature for future release
Major feature for next release
Severe bug fixed for production: hotfix 0.2
Incorporate bugfix in develop
Tag 0.2
Start of release branch for 1.0
From this point on, "next release" means the release after 1.0
Only bugfixes!
Bugfixes from rel. branch may be continuously merged back into develop
Tag 1.0

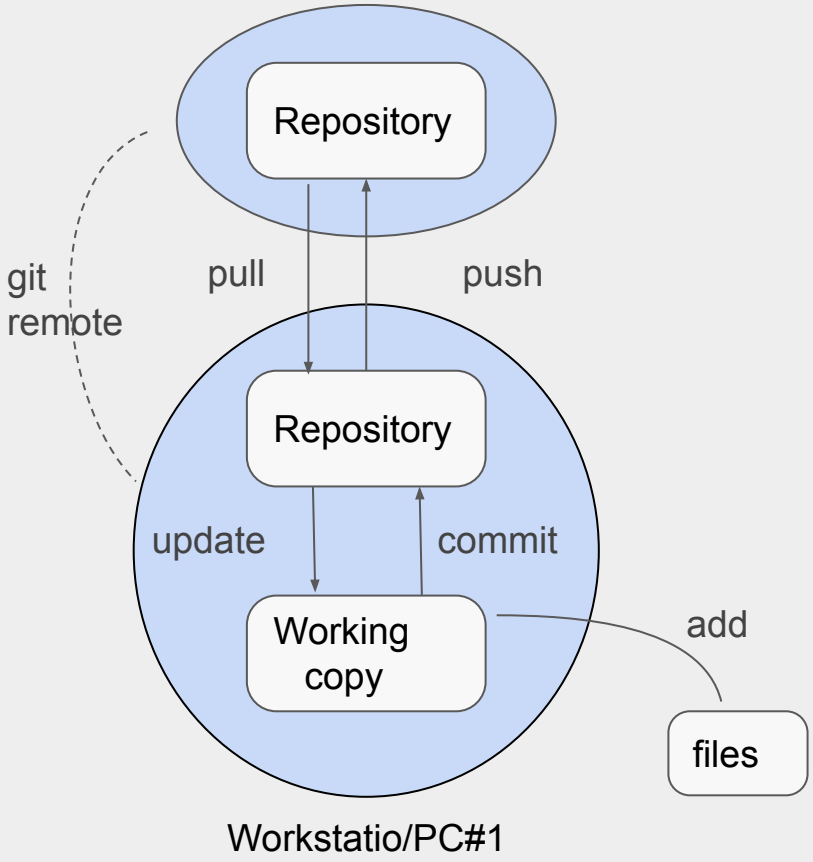

MASTER
v0.5
v1
Hotfix
v1.0
DEVELOP
FEATURE
FEATURE

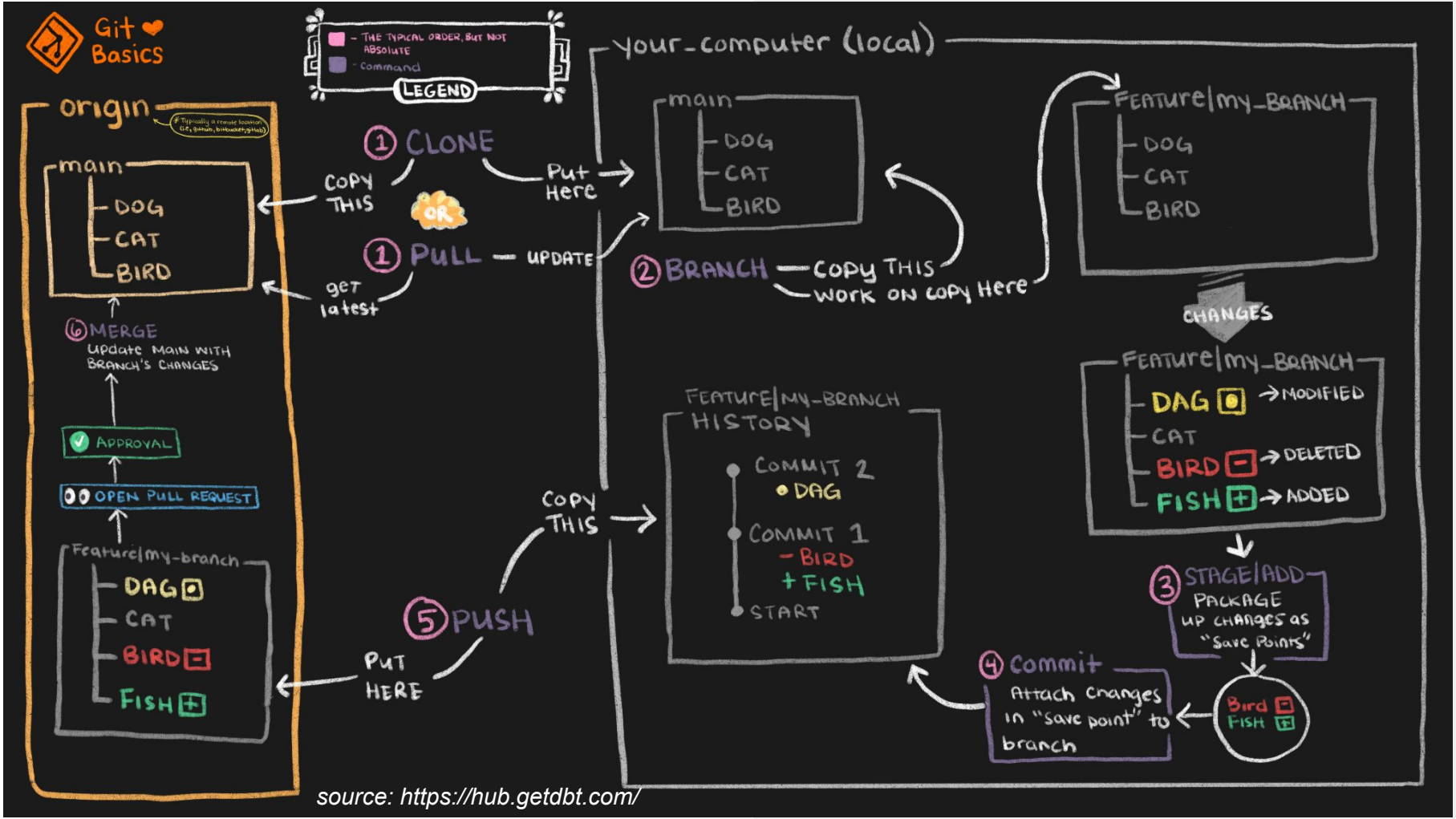
Pasos vincular un rep local -> remoto

1> Crear repositorio remoto en gitlab, github

2> Creamos un directorio local
3> entramos al directorio e inicializamos el git (**git init**) configuramos (**git config --list**)
4> vinculamos el repositorio local al remoto (**git remote add origin <url>**)
5> actualizamos el repositorio local (**git pull**)
6> cambiamos a la rama q vamos a trabajar (**git checkout main**)

7> agregamos nuestras modificaciones al área de trabajo (**git add <files>**)
8> guardamos un punto de restauración en nuestro repositorio con un comentario. (git commit -m “mensaje”)
9> …
10> Actualizamos el repositorio remoto. (**git push <url>**)





source: https://hub.getdbt.com/

Resumen comandos para bash:

1. `> cd <directorio>` - Cambiar el directorio actual.
2. `ls` - Listar los archivos y directorios en el directorio actual.
3. `pwd` - Mostrar la ruta del directorio actual.
4. `mkdir <nombre_directorio>` - Crear un nuevo directorio.
5. `rm <archivo>` - Eliminar un archivo.
6. `rm -r <directorio>` - Eliminar un directorio y su contenido de forma recursiva.
7. `cp <archivo_origen> <archivo_destino>` - Copiar un archivo.
8. `cp -r <directorio_origen> <directorio_destino>` - Copiar un directorio y su contenido de forma recursiva.
9. `mv <archivo_origen> <archivo_destino>` - Mover o renombrar un archivo.
10. `mv <directorio_origen> <directorio_destino>` - Mover o renombrar un directorio.
11. `cat <archivo>` - Mostrar el contenido de un archivo.
12. `echo <texto>` - Imprimir un texto en la salida estándar. (echo "" > archivo.txt)
13. `touch` > archivo.txt

Resumen comandos para git:

1. `git init` - Inicializar un repositorio Git.
2. `git clone <URL>` - Clonar un repositorio remoto.

1. `git status` - Ver el estado actual del repositorio.
2. `git add <archivo>` - Agregar un archivo específico a la zona de preparación (staging area).
3. `git add .` - Agregar todos los archivos modificados y eliminados a la zona de preparación (staging area).
4. `git commit -m "<mensaje>"` - Confirmar los cambios agregados en la zona de preparación con un mensaje descriptivo.
5. `git push` - Enviar los cambios confirmados al repositorio remoto.
6. `git pull` - Obtener los cambios más recientes del repositorio remoto y fusionarlos con tu rama local.
7. `git branch` - Ver las ramas disponibles y resaltar la rama actual.
8. `git checkout <rama>` - Cambiar a una rama específica.
9. `git merge <rama>` - Fusionar una rama específica con la rama actual.
10. `git log` - Ver el historial de confirmaciones (commits).
11. `git diff` - Mostrar las diferencias entre los cambios sin confirmar y el último commit.
12. `git reset <archivo>` - Deshacer los cambios realizados en un archivo específico.

Hasta la próxima

UNRN Universidad Nacional de **Río Negro**

UNRN Universidad Nacional de **Río Negro**

@unrionegro

unrn.edu.ar